

Reporte de Pruebas Técnicas - BIA Energy

Desarrollador: Carlos Mario Martinez Gongora

17 de agosto de 2026

Resumen

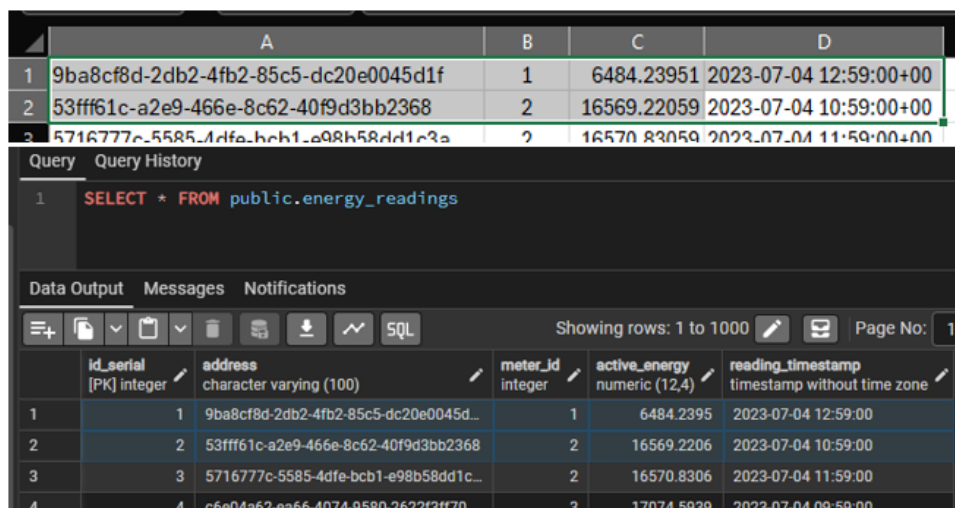
Este documento detalla la validación técnica del microservicio de agregación de series temporales de consumo eléctrico desarrollado en Golang bajo el enfoque de Clean Architecture. Se evidencia la integridad de los datos desde la extracción del archivo .csv original hasta la exposición de las métricas mediante la API HTTP REST.

1. Carga de Datos: .csv a PostgreSQL

El primer paso consistió en la migración de los datos crudos extraídos del archivo .csv suministrado hacia la base de datos relacional PostgreSQL.

- **Tabla creada:** energy_readings
- **Estructura mapeada:** id_serial, address, meter_id, active_energy, reading_timestamp.

La siguiente imagen evidencia la tabla en PostgreSQL completamente cargada con los registros históricos, lista para ser consumida por el microservicio en Go:



	A	B	C	D
1	9ba8cf8d-2db2-4fb2-85c5-dc20e0045d1f	1	6484.23951	2023-07-04 12:59:00+00
2	53fff61c-a2e9-466e-8c62-40f9d3bb2368	2	16569.22059	2023-07-04 10:59:00+00
3	5716777c-5585-4dfe-bcb1-e98b58dd1c3a	2	16570.83059	2023-07-04 11:59:00+00

	id_serial [PK] integer	address character varying (100)	meter_id integer	active_energy numeric (12,4)	reading_timestamp timestamp without time zone
1	1	9ba8cf8d-2db2-4fb2-85c5-dc20e0045d...	1	6484.2395	2023-07-04 12:59:00
2	2	53fff61c-a2e9-466e-8c62-40f9d3bb2368	2	16569.2206	2023-07-04 10:59:00
3	3	5716777c-5585-4dfe-bcb1-e98b58dd1c...	2	16570.8306	2023-07-04 11:59:00
4	4	c6e04a62-ea66-4074-9580-2622f3ff70...	3	17074.5939	2023-07-04 09:59:00

Figura 1: Contraste de datos: CSV original vs. PostgreSQL local(REGISTROS INICIALES)

22174	872241ac-bf4c-4264-96ed-f03a24270f69	1	303.82883	2022-08-24 16:57:05+00
22175	1be63fa4-1c32-475d-90a6-e85ccd2c834e	1	303.0769	2022-08-24 15:59:44+00
22176	53b9e101-176c-4cb9-a3d7-020277f46d78	1	302.31982	2022-08-24 14:52:19+00
22173	276fda18-0893-4c77-8e6d-3f72378ea6...	1	304.7072	2022-08-24 17:54:07
22174	872241ac-bf4c-4264-96ed-f03a24270f...	1	303.8288	2022-08-24 16:57:05
22175	1be63fa4-1c32-475d-90a6-e85ccd2c83...	1	303.0769	2022-08-24 15:59:44
22176	53b9e101-176c-4cb9-a3d7-020277f46...	1	302.3198	2022-08-24 14:52:19

Figura 2: Contraste de datos: CSV original vs. PostgreSQL local (REGISTROS FINAES)

2. Validación de Endpoints y Filtros (JSON)

Se ejecutaron pruebas integrales sobre el endpoint HTTP GET /consumption verificando la respuesta de la lógica de negocio frente a las diferentes frecuencias temporales (kind_period).

2.1. Prueba 1: Frecuencia Mensual (Monthly)

Se validó la sumatoria del consumo total agrupado por mes calendario para medidores específicos.

- **Query HTTP:**

end_date=2023-07-10&kind_period=monthly'?meters_ids=1&start_date=2023-06-01&end_date=2023-07-10&kind_period=monthly'

Contraste Exitoso: La sumatoria de active_energy de todos los días de agosto en el CSV coincide matemáticamente de forma exacta con la propiedad data_data devuelta en la caja JSON del microservicio.

```

localhost:8080/consumption?meters_ids=1&start_date=2023-06-01&end_date=2023-07-10&kind_period=monthly

Dar formato al texto ☒

{
  "period": [
    "JUN 2023",
    "JUL 2023"
  ],
  "data_graph": [
    {
      "meter_id": 1,
      "address": "Dirección del medidor 1",
      "active": [4401580.2204, 789036.9799],
      "reactive_inductive": [0, 0],
      "reactive_capacitive": [0, 0],
      "exported": [0, 0]
    }
  ]
}

```

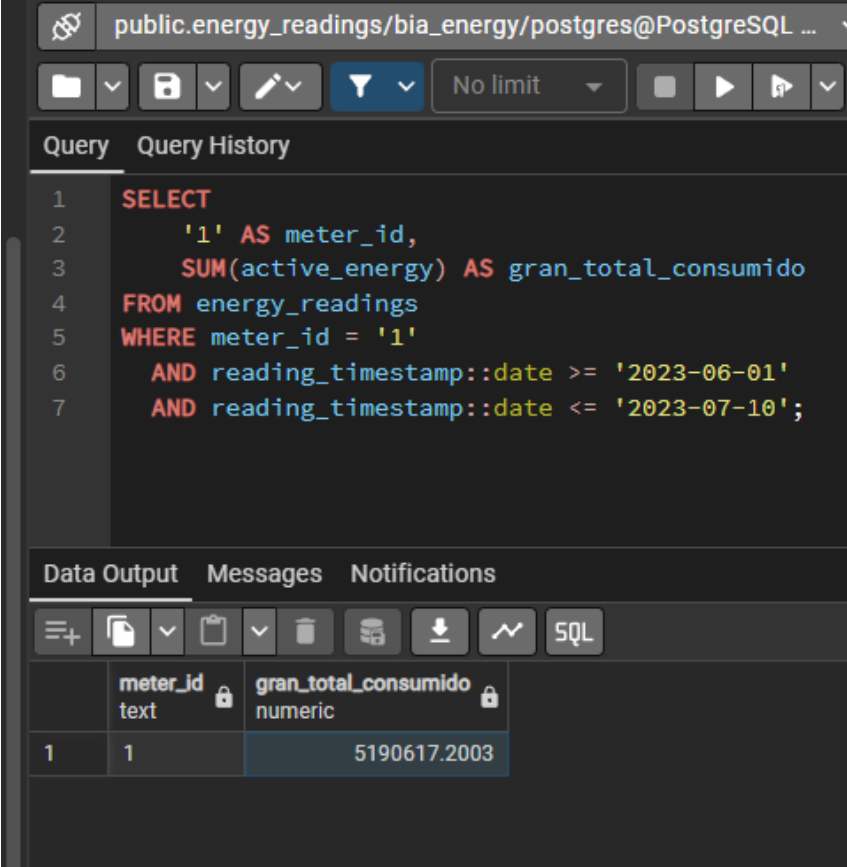
Figura 3: Respuesta HTTP JSON estructurada para consumo mensual.

Comprobación de los datos:

Para validar que el microservicio funciona perfectamente, sumamos los valores obtenidos en el JSON devuelto por el servidor HTTP: `.active`: [4401580.2204, 789036.9799].

Al sumar estos datos, el resultado es exactamente el mismo gran total que nos arroja la consulta directa en la base de datos PostgreSQL (como se ve en la captura anterior). Esto demuestra que el servidor lee, filtra y suma los consumos de manera correcta.

$$4.401.580,2204 + 789.036,9799 = 5.190.617,2003$$



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection to 'public.energy_readings/bia_energy/postgres@PostgreSQL ...'. Below the toolbar, the 'Query' tab is active, displaying the following SQL query:

```
1 SELECT
2     '1' AS meter_id,
3     SUM(active_energy) AS gran_total_consumido
4 FROM energy_readings
5 WHERE meter_id = '1'
6     AND reading_timestamp::date >= '2023-06-01'
7     AND reading_timestamp::date <= '2023-07-10';
```

The 'Data Output' tab is also visible, showing the results of the query in a table format:

	meter_id text	gran_total_consumido numeric
1	1	5190617.2003

Figura 4: Prueba Total de consumo al mes en PGSQL

2.2. Prueba 2: Frecuencia Semanal (Weekly)

Las series temporales se segmentaron correctamente utilizando la lógica de negocio para determinar los inicios de semana, agrupando bloques de 7 días.

- **Query HTTP:**

```
kind_period=weekly?meters_ids=1&start_date=2023-06-01&end_date=2023-06-26&
kind_period=weekly
```

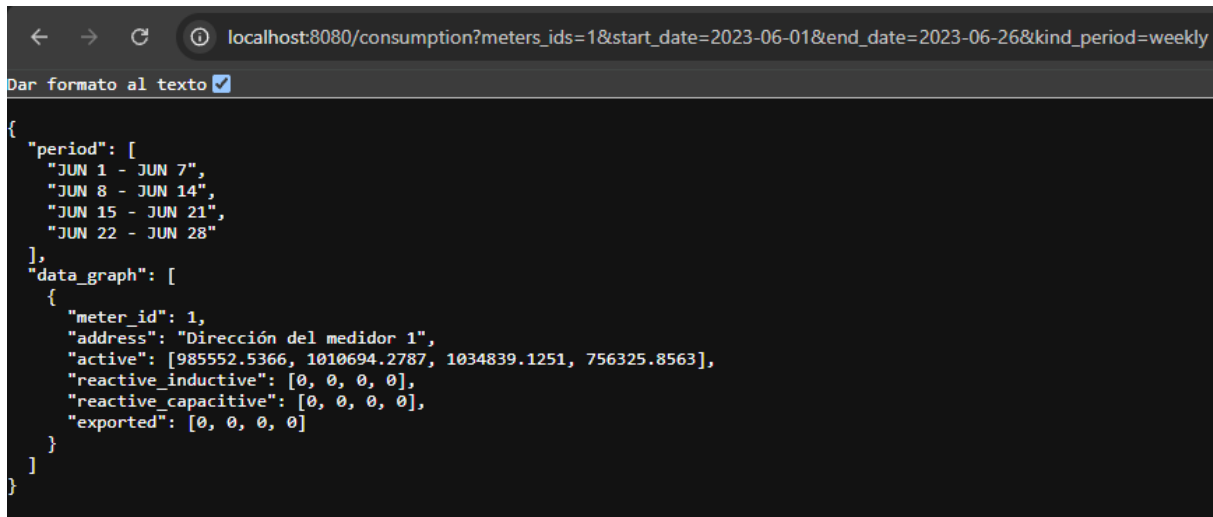


Figura 5: Respuesta HTTP JSON estructurada para consumo semanal.

Comprobación de los datos:

Para validar que el microservicio funciona perfectamente, sumamos los valores obtenidos en el JSON devuelto por el servidor HTTP: `.active`: [985552.5366, 1010694.2787, 1034839.1251, 756325.8563].

Al sumar estos datos, el resultado es exactamente el mismo gran total que nos arroja la consulta directa en la base de datos PostgreSQL (como se ve en la captura anterior). Esto demuestra que el servidor lee, filtra y suma los consumos de manera correcta.

$$985.552,5366 + 1.010.694,2787 + 1.034.839,1251 + 756.325,8563 = 3.787.411,7967$$

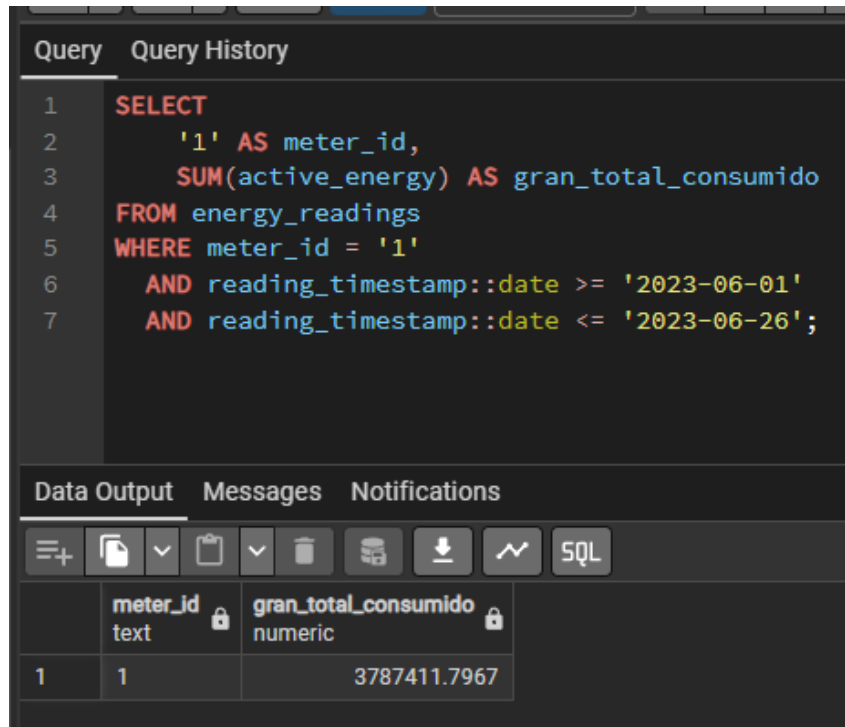


Figura 6: Prueba Total de consumo a la semana en PGSQL

2.3. Prueba 3: Frecuencia Diaria (Daily)

La granularidad más detallada de las series temporales arrojó resultados iterados día por día.

■ Query HTTP:

kind_period=daily?meters_ids=1,2,3&start_date=2023-06-01&end_date=2023-06-10

kind_period=daily

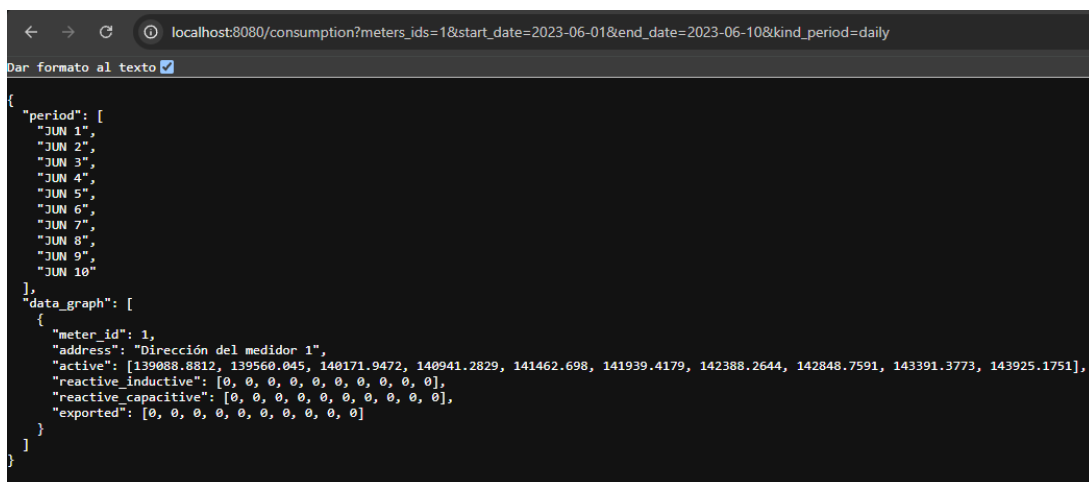


Figura 7: Respuesta HTTP JSON estructurada para consumo diario.

Comprobación de los datos:

Para validar que el microservicio funciona perfectamente, sumamos los valores obtenidos en el JSON devuelto por el servidor HTTP:

Al sumar estos datos, el resultado es exactamente el mismo gran total que nos arroja la consulta directa en la base de datos PostgreSQL (como se ve en la captura anterior). Esto demuestra que el servidor lee, filtra y suma los consumos de manera correcta.

$$\begin{aligned} &139.088,8812 + 139.560,0450 + 140.171,9472 + 140.941,2829 + 141.462,6980 + \\ &141.939,4179 + 142.388,2644 + 142.848,7591 + 143.391,3773 + 143.925,1751 \\ &= 1.415.717,8481. \end{aligned}$$

The screenshot shows a PostgreSQL query editor interface. The top section, titled 'Query', contains a SQL query. The bottom section, titled 'Data Output', shows the results of the query in a table format. The table has two columns: 'meter_id' (text) and 'gran_total_consumido' (numeric). The results show a single row with 'meter_id' equal to 1 and 'gran_total_consumido' equal to 1415717.8481.

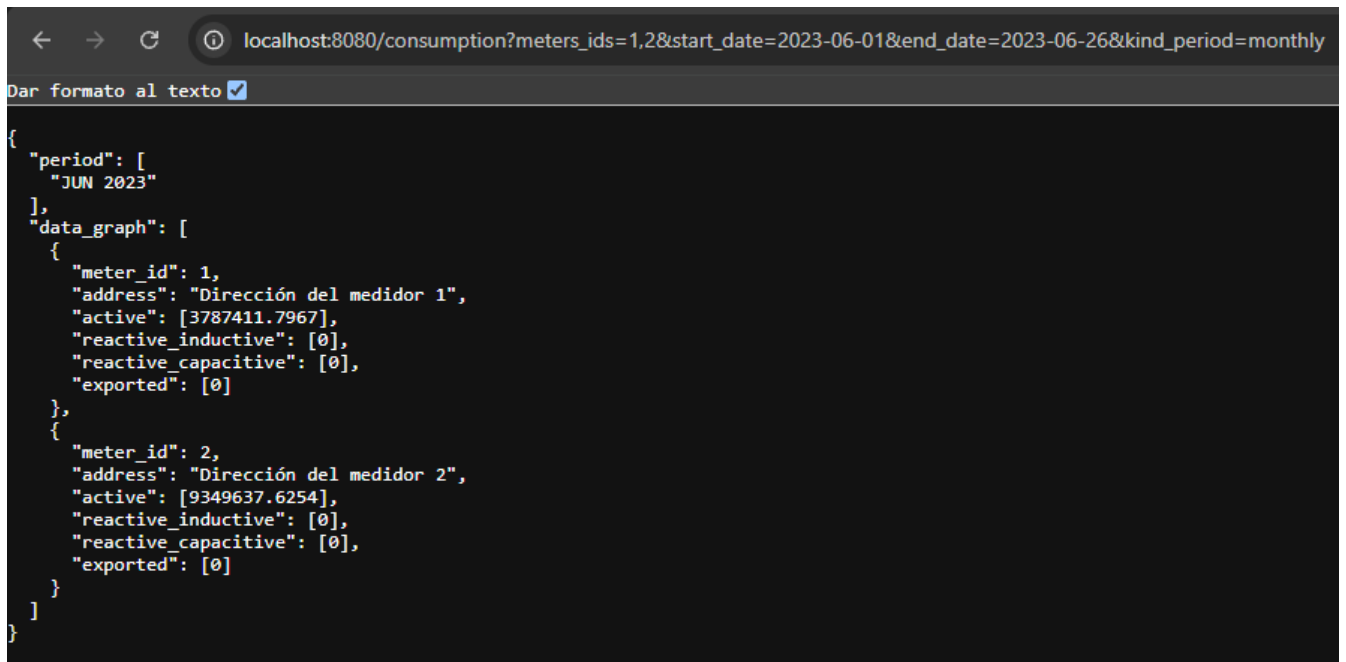
```
1 SELECT
2     '1' AS meter_id,
3     SUM(active_energy) AS gran_total_consumido
4 FROM energy_readings
5 WHERE meter_id = '1'
6     AND reading_timestamp::date >= '2023-06-01'
7     AND reading_timestamp::date <= '2023-06-10';
```

	meter_id text	gran_total_consumido numeric
1	1	1415717.8481

Figura 8: Prueba total del consumo diario en PostgreSQL.

2.4. Prueba 4: Integracion de mas de un meter id

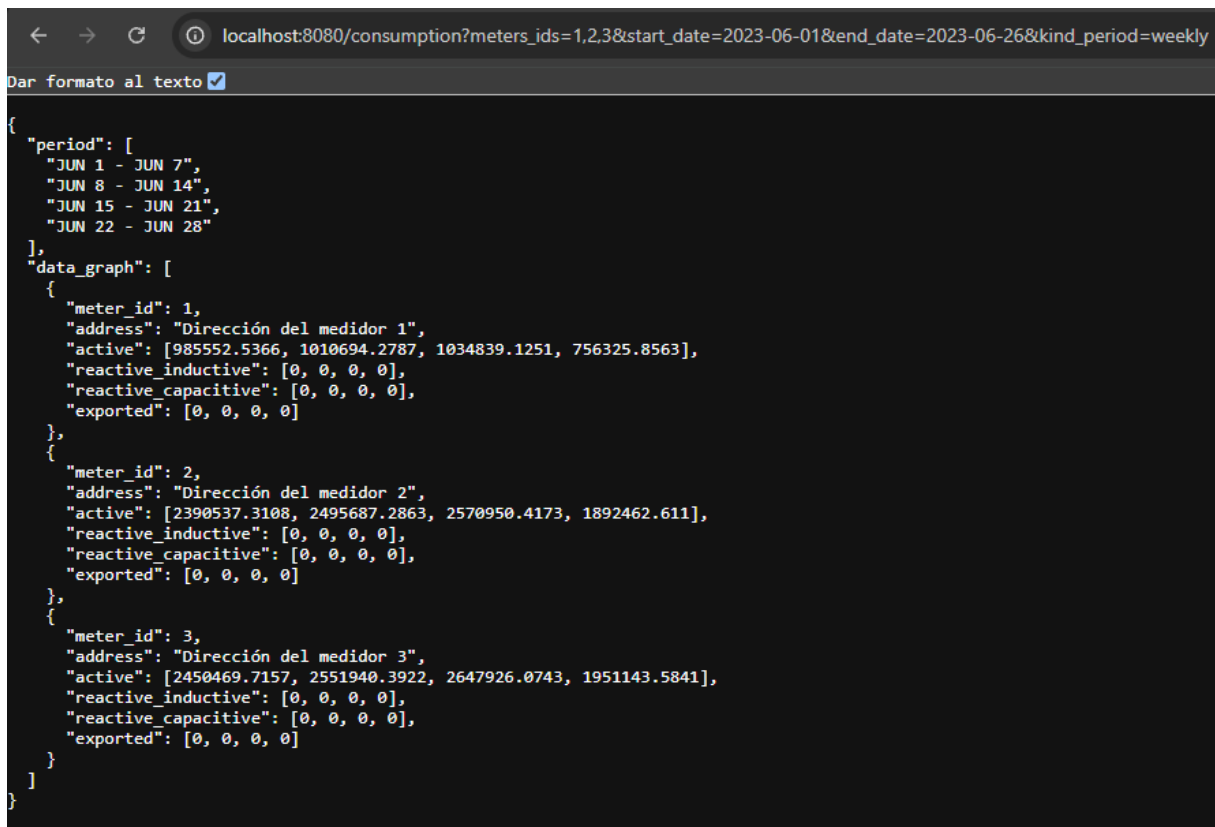
Acontinuacion se realizaron pruebas ejecutando el programa con mas de un ID, los resultados fueron los siguientes:



```
localhost:8080/consumption?meters_ids=1,2&start_date=2023-06-01&end_date=2023-06-26&kind_period=monthly

{
  "period": [
    "JUN 2023"
  ],
  "data_graph": [
    {
      "meter_id": 1,
      "address": "Dirección del medidor 1",
      "active": [3787411.7967],
      "reactive_inductive": [0],
      "reactive_capacitive": [0],
      "exported": [0]
    },
    {
      "meter_id": 2,
      "address": "Dirección del medidor 2",
      "active": [9349637.6254],
      "reactive_inductive": [0],
      "reactive_capacitive": [0],
      "exported": [0]
    }
  ]
}
```

Figura 9: Implementacion de Id:1,2 para periodo mensual



```
localhost:8080/consumption?meters_ids=1,2,3&start_date=2023-06-01&end_date=2023-06-26&kind_period=weekly

{
  "period": [
    "JUN 1 - JUN 7",
    "JUN 8 - JUN 14",
    "JUN 15 - JUN 21",
    "JUN 22 - JUN 28"
  ],
  "data_graph": [
    {
      "meter_id": 1,
      "address": "Dirección del medidor 1",
      "active": [985552.5366, 1010694.2787, 1034839.1251, 756325.8563],
      "reactive_inductive": [0, 0, 0, 0],
      "reactive_capacitive": [0, 0, 0, 0],
      "exported": [0, 0, 0, 0]
    },
    {
      "meter_id": 2,
      "address": "Dirección del medidor 2",
      "active": [2390537.3108, 2495687.2863, 2570950.4173, 1892462.611],
      "reactive_inductive": [0, 0, 0, 0],
      "reactive_capacitive": [0, 0, 0, 0],
      "exported": [0, 0, 0, 0]
    },
    {
      "meter_id": 3,
      "address": "Dirección del medidor 3",
      "active": [2450469.7157, 2551940.3922, 2647926.0743, 1951143.5841],
      "reactive_inductive": [0, 0, 0, 0],
      "reactive_capacitive": [0, 0, 0, 0],
      "exported": [0, 0, 0, 0]
    }
  ]
}
```

Figura 10: Implementacion de Id:1,2,3 para periodo semanal

